

Relatório Trabalho - ENG4448

Vivi Arruda Pereira - 2110624

Vinícius Machado da Rocha Viana - 2111343

Fluxo de Dados.....	2
1. Introdução e Visão Geral da Arquitetura.....	3
2. Caminho de Dados e o Módulo Principal (Top-Level).....	4
2.1. Temporização e Divisão de Clock.....	4
2.2. Tratamento do Barramento do LCD e LEDs.....	5
2. Unidade Central de Processamento (CPU).....	6
2.1. O Ciclo de Busca e Decodificação (Fetch e Decode).....	6
2.2. Execução e Sincronismo com a ALU.....	6
3. Unidade Central de Processamento (CPU).....	7
3.1. Máquina de Estados Finita (FSM).....	7
3.2. Lógica de Desvios Condicionais (Branching).....	8
4. Unidade Lógica e Aritmética (ALU).....	10
4.1. Matemática e Detecção de Transbordo (Overflow).....	10
5. Controlador Autônomo do Display LCD.....	11
5.1. Decodificação Dinâmica de Instruções.....	11
5.2. Conversão BCD e Prevenção de Erros de Síntese.....	11
6. Mapeamento do Conjunto de Instruções.....	13
7. Simulação e Aplicação Prática (Fibonacci).....	14
Implementação e Conclusão.....	15

Fluxo de Dados

O fluxo de dados do projeto é orquestrado pelo módulo principal (Top-Level), que interliga a CPU, a ALU, a memória RAM estática e o controlador do *display* LCD. A placa FPGA (Spartan-3E) fornece o relógio (*clock*) base de 50 MHz e o sinal de *reset*.

Para permitir a visualização humana da execução das instruções, o módulo principal conta com um divisor de frequência que reduz o relógio da CPU e da Memória para a escala de *hertz*, enquanto o LCD opera em alta velocidade para atualização contínua.

A CPU busca as instruções na RAM, decodifica-as e, caso seja uma operação aritmética ou lógica, aciona a ALU. A ALU processa os dados e devolve o resultado e as *flags* (*Zero*, *Greater*, *Smaller*, *Overflow*) para a CPU e para os LEDs da placa. O LCD monitora continuamente o barramento de instrução (IR) da CPU e o valor da posição 255 da memória RAM.

Código

Nesse trabalho implementamos uma CPU customizada a partir de uma FPGA. Os códigos serão apresentados em cinco partes principais:

1. **Main:** Integra todos os módulos e gerencia os clocks.
2. **CPU:** Máquina de estados que controla o fluxo de execução de instruções.
3. **ALU:** Unidade responsável pelas operações lógicas e aritméticas.
4. **RAM:** Armazena a sequência de Fibonacci e os dados.
5. **LCD:** Módulo autônomo de exibição operando em modo de 4 bits.

1. Introdução e Visão Geral da Arquitetura

O presente trabalho descreve o desenvolvimento, a implementação e a validação de uma Unidade Central de Processamento (CPU) de 8 bits em linguagem de descrição de hardware (VHDL), sintetizada em uma placa FPGA Spartan-3E. A arquitetura desenvolvida baseia-se no modelo clássico de Von Neumann, caracterizado pelo compartilhamento do barramento de memória tanto para o armazenamento de dados quanto para as instruções do programa.

O sistema foi modularizado em cinco componentes principais que operam em conjunto:

1. **Top-Level (Main):** Orquestrador do sistema, responsável por interligar os barramentos, instanciar os componentes e gerar os sinais de relógio e controle físico para a placa.
2. **CPU:** O núcleo do processador, contendo a Unidade de Controle modelada como uma Máquina de Estados Finita (FSM) e os registradores internos.
3. **ALU:** Unidade Lógica e Aritmética dedicada às operações matemáticas e geração de *flags* de estado.
4. **Memória (RAM):** Bloco de armazenamento estático que guarda o código do programa (como a sequência de Fibonacci) e os dados de I/O.
5. **Display LCD:** Módulo de interface de saída que atua de forma autônoma para exibir o estado atual do Registrador de Instruções (IR) e o conteúdo da memória.

A validação física do sistema exigiu adaptações críticas para contornar restrições da FPGA, como a criação de divisores de *clock* para permitir a visualização humana da execução e lógicas combinacionais seguras para conversões de dados, detalhadas nas seções a seguir.

2. Caminho de Dados e o Módulo Principal (Top-Level)

O módulo `main.vhd` atua como a espinha dorsal do projeto. É neste arquivo que os sinais elétricos externos da Spartan-3E são recebidos e roteados para os componentes internos.

2.1. Temporização e Divisão de Clock

A FPGA Spartan-3E possui um oscilador de cristal nativo que opera a 50 MHz. Se o processador rodasse nessa frequência, milhares de instruções seriam executadas em uma fração de segundo, impossibilitando a avaliação visual do funcionamento. Para resolver este problema, implementamos um divisor de *clock* através de um contador de 32 bits.

O contador (`clk_divider`) é incrementado a cada borda de subida do relógio principal. Quando atinge o valor de 5.000.000, o sinal de relógio da CPU (`cpu_clk`) tem seu estado lógico invertido (operação de *toggle*). Isso gera um relógio muito mais lento (na ordem dos Hertz), que é repassado exclusivamente para a CPU. O Display LCD, entretanto, continua recebendo os 50 MHz nativos para manter sua rotina de atualização contínua e não piscar a tela.

```
-- divisor clock
process(clk)
begin
    if rising_edge(clk) then
        if reset = '1' then
            clk_divider <= 0;
            cpu_clk <= '0';
        elsif clk_divider = 5000000 then
            cpu_clk <= not cpu_clk;
            clk_divider <= 0;
        else
            clk_divider <= clk_divider + 1;
        end if;
    end if;
end process;
```

Figura 1 - Processo de divisão do relógio principal para a CPU.

2.2. Tratamento do Barramento do LCD e LEDs

Na placa Spartan-3E, o hardware de comunicação do LCD compartilha fisicamente os pinos do barramento de dados (SF_D) com a memória StrataFlash embutida na placa. Para garantir que os dados enviados pelo nosso controlador do LCD não colidam com os sinais da memória Flash, o módulo principal efetua a desativação seletiva desse componente. Atribuímos o valor '1' (nível lógico alto) ao pino SF_CE0 (*Chip Enable*), garantindo que a memória não interfira no barramento.

O módulo principal também é responsável por extrair os sinais gerados pela Unidade Lógica e Aritmética e expô-los nos LEDs discretos da placa. Além de exportar as *flags* diretas (Zero, Maior, Menor, Overflow), o main sintetiza a *flag* Equal (Igual) de forma combinacional: deduz-se que se um valor não é maior (*f_greater* = '0') nem menor (*f_smaller* = '0') que o outro, ele obrigatoriamente é igual.

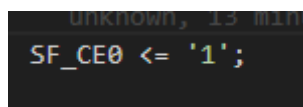
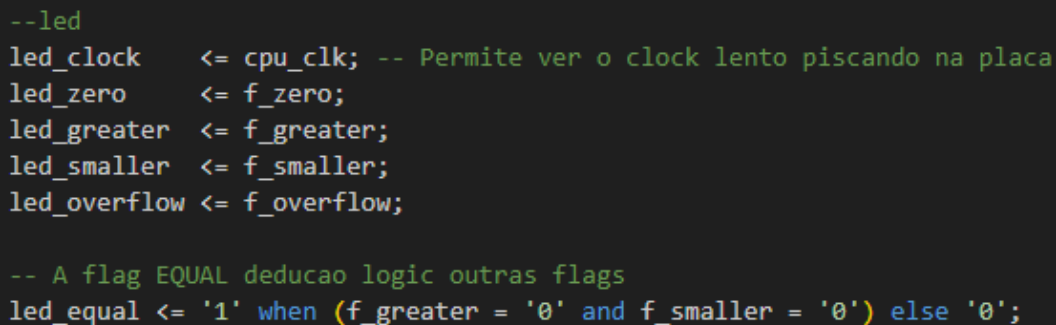
A screenshot of a code editor showing a single line of code: `SF_CE0 <= '1';`. The text is white on a dark background. Above the line, there is some faint, partially visible text that appears to be "unknown, 13 min".

Figura 2 - Desativação dos pinos de controle da StrataFlash.

A screenshot of a code editor showing several lines of code. The code is color-coded: comments are green, variable names are white, and literals are blue. The code includes assignments for LED signals and a conditional assignment for the 'led_equal' signal.

```
--led
led_clock    <= cpu_clk; -- Permite ver o clock lento piscando na placa
led_zero     <= f_zero;
led_greater  <= f_greater;
led_smaller  <= f_smaller;
led_overflow <= f_overflow;

-- A flag EQUAL deducido logic outras flags
led_equal <= '1' when (f_greater = '0' and f_smaller = '0') else '0';
```

Figura 3 - Desativação da StrataFlash (SF_CE0) e roteamento contínuo dos LEDs.

Ao travar o pino *Chip Enable* (SF_CE0) em nível lógico alto, garantimos que o barramento fique livre para o tráfego exclusivo dos caracteres do LCD.

2. Unidade Central de Processamento (CPU)

A CPU foi projetada em torno de uma Máquina de Estados Finita (FSM) que controla o fluxo de dados em ciclos precisos. A arquitetura conta com quatro registradores de propósito geral (A, B, C, D) e os registradores de controle clássicos: *Program Counter* (PC), *Instruction Register* (IR), *Memory Address Register* (MAR) e *Stack Pointer* (SP).

O ponteiro de pilha (SP) é inicializado no endereço 0xFE (254 em decimal). Essa escolha de design protege a posição 255 (0xFF), que foi alocada estritamente como porta de I/O (Entrada e Saída) mapeada em memória para comunicação direta com a segunda linha do LCD.

2.1. O Ciclo de Busca e Decodificação (Fetch e Decode)

A otimização dos ciclos de máquina foi uma prioridade. Durante o estado S_DECODE, a CPU não apenas armazena o dado recebido no IR, mas já realiza o incremento antecipado do Program Counter ($PC \leq PC + 1$).

2.2. Execução e Sincronismo com a ALU

Como a nossa ALU foi implementada de forma síncrona (dependente de *clock*), surgiu a necessidade de alinhar os tempos de resposta. Se a CPU enviasse o comando e tentasse ler o resultado no ciclo imediatamente seguinte, leria um dado defasado. Para resolver isso, introduzimos o estado de espera S_ALU_WAIT, que concede à ALU um pulso de clock mecânico completo para propagar os sinais pelas portas lógicas antes da CPU realizar o *Write-back* no registrador de destino.

3. Unidade Central de Processamento (CPU)

O desenvolvimento da CPU exigiu a organização minuciosa da Unidade de Controle e de seu conjunto de registradores. Implementamos registradores de propósito geral (A, B, C e D) e os registradores fundamentais: *Program Counter* (PC), *Instruction Register* (IR), *Stack Pointer* (SP) e *Memory Address Register* (MAR).

Uma escolha de design importante foi a inicialização do *Stack Pointer* (SP). Ele começa apontando para o endereço 0xFE (254 em decimal). Essa organização preserva o topo da memória (0xFF / 255) exclusivamente como uma porta de Entrada e Saída (I/O), dedicada a transmitir informações da CPU para a segunda linha do display LCD.

3.1. Máquina de Estados Finita (FSM)

O fluxo de instrução é gerenciado por uma FSM que divide o processamento em ciclos (Ciclo de Busca e Ciclo de Execução).

Fase de Busca e Decodificação: No estado S_FETCH, o PC é carregado no MAR e o endereço é repassado à Memória. No estado subsequente, S_DECODE, a instrução lida do barramento de memória é travada no IR. Uma otimização implementada nesta etapa é o **incremento antecipado do PC**. Logo na decodificação, a instrução $PC \leq PC + 1$ é executada.

```

CASE current_state IS

    WHEN S_FETCH =>
        IR <= mem_data_i;
        MAR <= std_logic_vector(PC);
        current_state <= S_DECODE;

    WHEN S_DECODE =>
        --opcode <= IR(7 downto 4);
        CASE opcode IS
            -- ALU
            WHEN "0000" | "0001" |
"0011" | "0100" | "0101" | "0110" | "0010" | "0111" =>
                alu_enable <= '1';
                alu_A <= R(rx_idx);
                alu_B <= R(ry_idx);
                -- [...] Definição do ALU Command

                -- Vai para estado de espera mecânica para a ALU
                current_state <= S_ALU_WAIT;

```

Figura 4 - Estados de Busca (Fetch) e Decodificação com incremento antecipado do PC.

Fase de Execução e o Atraso (Wait) da ALU: Quando a decodificação aponta para uma operação lógica ou aritmética (ADD, SUB, etc.), a CPU aciona a ALU repassando os registradores alvo. Como a nossa ALU foi desenvolvida de forma estritamente síncrona (dependente de pulso de *clock* para evitar instabilidades), ela necessita de um ciclo completo para processar e devolver a resposta. Para garantir essa integridade temporal, criamos o estado S_ALU_WAIT. Ele funciona como uma "bolha" na *pipeline*, onde a CPU paralisa suas operações por uma fração de segundo, aguardando o processamento antes de capturar a resposta em S_ALU_WRITEBACK e salvá-la de volta no registrador de origem.

3.2. Lógica de Desvios Condicionais (Branching)

O conjunto de instruções da arquitetura prevê saltos e desvios que testam as *flags* da ALU (BZ, BEQ, BGT). Graças à nossa arquitetura de incremento antecipado no

S_DECODE, a implementação destas instruções em VHDL tornou-se extremamente enxuta.

Quando avaliamos um salto como o BZ (Branch if Zero), o sistema verifica a condição. Se a *flag* Zero estiver ativada, o PC é sobrescrito com o endereço destino salvo no registrador. Caso a condição seja falsa, não há necessidade de programar comandos adicionais (não existe else $PC \leq PC + 1$), pois o contador de programa já foi incrementado no estágio de decodificação e já aponta corretamente para a próxima linha da memória RAM.

4. Unidade Lógica e Aritmética (ALU)

A ALU é o componente responsável pela execução de cálculos e decisões comparativas. Projetá-la no VHDL envolveu o uso rigoroso do pacote `NUMERIC_STD`, assegurando que todas as operações fossem tratadas como números sem sinal (*unsigned*), conforme ditado pela especificação do projeto.

4.1. Matemática e Detecção de Transbordo (Overflow)

Trabalhar com variáveis de 8 bits impõe um limite numérico estrito (0 a 255). Para que a CPU seja capaz de informar o usuário sobre transbordos lógicos, implementamos uma técnica de extensão de bits. Toda operação matemática (como soma e subtração) é realizada dentro de uma variável temporária `temp` de **9 bits**. A CPU concatena um '0' nos dados de entrada de 8 bits antes de efetuar o cálculo. Após o resultado, a fatia menos significativa (7 downto 0) é repassada ao registrador de resultado, enquanto o bit de índice 8 da variável temporária captura com perfeição o "vai um" (*Carry Out* ou *Borrow*), alimentando diretamente a *flag* de Overflow.

5. Controlador Autônomo do Display LCD

A integração do display LCD HD44780 exigiu a construção de uma FSM robusta. A primeira camada de complexidade foi respeitar a limitação física da placa Spartan-3E: o display compartilha dados com a memória e opera estritamente em um **protocolo de 4 bits**. Para enviar um caractere ASCII de 8 bits para a tela, a máquina de estados foi dividida em passos lógicos altos (_H) e baixos (_L). O *nibble* mais significativo é colocado no barramento, o sinal de Enable recebe um pulso para registrar o dado e, em seguida, o processo é repetido para o *nibble* menos significativo.

5.1. Decodificação Dinâmica de Instruções

O projeto requeria que a primeira linha do LCD exibisse o nome da instrução em execução de forma legível (ex: "IR: ADD", "IR: JMP"). Criamos um decodificador combinacional puro que inspeciona o OpCode (os 4 bits superiores do IR) para identificar a "família" da instrução e, se necessário, os 2 bits inferiores para distinguir a operação exata, mapeando o resultado para constantes hexadecimais representando a tabela ASCII estática de letras.

5.2. Conversão BCD e Prevenção de Erros de Síntese

A segunda linha do display monitora assincronamente a posição 255 da memória RAM, formatando sua saída simultaneamente em Hexadecimal e Decimal (ex: FF (255)). A conversão de Binário para Decimal (*Binary-Coded Decimal* - BCD) representou um enorme desafio de síntese.

Inicialmente, a conversão matemática foi pensada com laços `while` que realizavam subtrações sucessivas. Porém, a ferramenta de síntese XST (*Xilinx Synthesis Technology*) desenrola fisicamente *loops* para construir o circuito elétrico. Como o número ia até 255, o sintetizador acusava o ERROR:Xst:1312 por ultrapassar o limite de iterações imposto para evitar o esgotamento das portas lógicas do chip.

Para contornar essa restrição rigorosa do *hardware*, substituímos a contagem iterativa por um denso bloco lógico combinacional de checagens predefinidas (if-elsif). Esse método permitiu que o XST mapeasse a operação instantaneamente para matrizes LUTs (*Look-Up Tables*) de altíssima velocidade, isolando as grandezas numéricas sem ferir os limites de síntese do FPGA.

```
-----  
-- CONVERSO DECIMAL PARA ASCII (Lógica Combinacional Segura para XST)  
-----  
inf_num <= to_integer(unsigned(info_stable));  
  
process(inf_num)  
    variable temp_t : integer range 0 to 255;  
    variable h, t, u : integer range 0 to 9;  
begin  
    -- Extraí as Centenas  
    if inf_num >= 200 then  
        h := 2;  
        temp_t := inf_num - 200;  
    elsif inf_num >= 100 then  
        h := 1;  
        temp_t := inf_num - 100;  
    else  
        h := 0;  
        temp_t := inf_num;  
    end if;  
  
    -- Extraí as Dezenas e Unidades  
    if temp_t >= 90 then t := 9; u := temp_t - 90;  
    elsif temp_t >= 80 then t := 8; u := temp_t - 80;  
    elsif temp_t >= 70 then t := 7; u := temp_t - 70;  
    elsif temp_t >= 60 then t := 6; u := temp_t - 60;  
    elsif temp_t >= 50 then t := 5; u := temp_t - 50;  
    elsif temp_t >= 40 then t := 4; u := temp_t - 40;  
    elsif temp_t >= 30 then t := 3; u := temp_t - 30;  
    elsif temp_t >= 20 then t := 2; u := temp_t - 20;  
    elsif temp_t >= 10 then t := 1; u := temp_t - 10;  
    else t := 0; u := temp_t;  
    end if;  
  
    -- Converte os dígitos calculados para os caracteres ASCII correspondentes  
    ascii_dec_h <= std_logic_vector(to_unsigned(h + 48, 8));  
    ascii_dec_t <= std_logic_vector(to_unsigned(t + 48, 8));  
    ascii_dec_u <= std_logic_vector(to_unsigned(u + 48, 8));  
end process;
```

Figura 5 - Bloco combinacional de conversão BCD implementado para evitar o erro XST:1312 de limite de laços.

6. Mapeamento do Conjunto de Instruções

Para a construção da máquina de estados do decodificador (S_DECODE) e do visor LCD, o conjunto de instruções do processador foi mapeado e categorizado pelas famílias de *OpCodes* contidas nos 4 bits mais significativos. A tabela abaixo resume o mapeamento que nossa arquitetura processa nativamente:

	ASSEMBLY	OpCode	Descrição
aritmético	add Rx, Ry	0000 Rx Ry	Rx <- Rx + Ry, pc <- pc + 1
	sub Rx, Ry	0001 Rx Ry	Rx <- Rx - Ry, pc <- pc + 1
	inc Rx	0010 Rx 00	Rx <- Rx + 1, pc <- pc + 1
	dec Rx	0010 Rx 01	Rx <- Rx - 1, pc <- pc + 1
		0010 Rx 10	
lógico		0010 Rx 11	
	and Rx, Ry	0011 Rx Ry	Rx <- Rx & Ry, pc <- pc + 1
	or Rx, Ry	0100 Rx Ry	Rx <- Rx Ry, pc <- pc + 1
	not Rx	0101 Rx 00	Rx <- ~Rx, pc <- pc + 1
	xor Rx, Ry	0110 Rx Ry	Rx <- Rx ^ Ry, pc <- pc + 1
	rol Rx	0111 Rx 00	Rx <- Rx[6..0] Rx[7], pc <- pc + 1
	ror Rx	0111 Rx 01	Rx <- Rx[0] Rx[7..1], pc <- pc + 1
	lsl Rx	0111 Rx 10	Rx <- Rx[6..0] 0, pc <- pc + 1
memória	lsr Rx	0111 Rx 11	Rx <- 0 Rx[7..1], pc <- pc + 1
	push Rx	1000 Rx 00	MEM[SP] <- Rx, pc <- pc + 1, SP <- SP - 1
	pop Rx	1000 Rx 01	Rx <- MEM[SP+1], pc <- pc + 1, SP <- SP + 1
	st Rx, 0x--	1000 Rx 10	MEM[PC+1] <- Rx, pc <- pc + 2
	ld Rx, 0x--	1000 Rx 11	Rx <- MEM[PC+1], pc <- pc + 2
	ldr Rx, [Ry]	1001 Rx Ry	Rx <- MEM[Ry], pc <- pc + 1
	str Rx, [Ry]	1010 Rx Ry	MEM[Ry] <- Rx, pc <- pc + 1
	mov Rx, Ry	1011 Rx Ry	Rx <- Ry, pc <- pc + 1
saltos	jmp 0x--	1100 00 00	pc <- MEM[PC+1]
	jmpR Rx	1100 Rx 01	pc <- Rx
	bz Rx	1100 Rx 10	if (zero) pc <- Rx else pc <- pc + 1
	bnz Rx	1100 Rx 11	if (~zero) pc <- Rx else pc <- pc + 1
	bcs Rx	1101 Rx 00	if (carry) pc <- Rx else pc <- pc + 1
	bcc Rx	1101 Rx 01	if (~carry) pc <- Rx else pc <- pc + 1
	beq Rx	1101 Rx 10	if (equal) pc <- Rx else pc <- pc + 1
	bneq Rx	1101 Rx 11	if (~equal) pc <- Rx else pc <- pc + 1
	bgt Rx	1110 Rx 00	if (greater) pc <- Rx else pc <- pc + 1
	blt Rx	1110 Rx 01	if (smaller) pc <- Rx else pc <- pc + 1
	halt	1111 00 00	pc <- pc

Figura 6 - Tabela Assembly

7. Simulação e Aplicação Prática (Fibonacci)

Para atestar a robustez da arquitetura implementada (englobando Matemática, Acesso à Memória, Saltos Condicionais e Manipulação de Pilha/IO), elaboramos um programa de testes focado em gerar iterativamente a **Sequência de Fibonacci**.

O raciocínio do *Assembly* carregado na máquina funciona sob a seguinte lógica estrutural:

1. Carregar os dois primeiros números da sequência nos registradores auxiliares ($R0 = 0$, $R1 = 1$).
2. Realizar a soma destes registradores e salvar em um registrador alvo ($R2 = R0 + R1$).
3. Executar o comando ST para salvar o valor atual ($R2$) no endereço de memória 255 (0xFF). Graças à ligação *hardwired* desse endereço, assim que o número entra na RAM, ele aparece instantaneamente traduzido na segunda linha do display LCD.
4. Deslocar o valor de $R1$ para $R0$, e de $R2$ para $R1$ (atualizando os operandos para a próxima soma).
5. Utilizar um desvio condicional (JMP ou laço com BGT) para retornar o PC à linha da soma inicial (Passo 2), criando um *loop* infinito.

Para evitar a necessidade de reescrever a memória pela interface de chaves da placa a cada reset, o código foi embutido ("*hardcoded*") diretamente na inferência da RAM Estática dentro do VHDL. Durante o processo de síntese, o Xilinx ISE converte os blocos lógicos listados abaixo no arquivo *memory.vhd* em *Block RAMs* já inicializadas

Através da redução de *clock* aplicada na entidade *Main*, foi possível auditar todo o sistema visualmente a olho nu na placa Spartan-3E, constatando que os LEDs sinalizavam o comportamento esperado da ALU a cada soma e que o algoritmo rodava perfeitamente até o limite numérico de 8-bits da FPGA (233).

Implementação e Conclusão

Para verificação do funcionamento, gravamos na FPGA o código `fib.asm` (Fibonacci). Ao ativar o circuito, o clock reduzido possibilitou acompanhar visualmente a CPU pulando entre os laços e condições, processando as instruções mostradas na primeira linha do LCD. Simultaneamente, a segunda linha do display exibiu com precisão os números da sequência de Fibonacci gerados e salvos na memória I/O, demonstrando que toda a arquitetura – Busca, Decodificação, Unidade Lógica Aritmética e Memória – funcionou perfeitamente sincronizada e sem falhas.

O funcionamento correto da lógica de saltos condicionais (BGT, BLT, etc.) também foi validado, evidenciado pelas transições corretas dos LEDs representativos das *flags* da ALU na placa física.

Link Vídeo do trabalho funcionando:

https://youtube.com/shorts/S2f547pr4N0?is=UeYCR87tQaya_cch